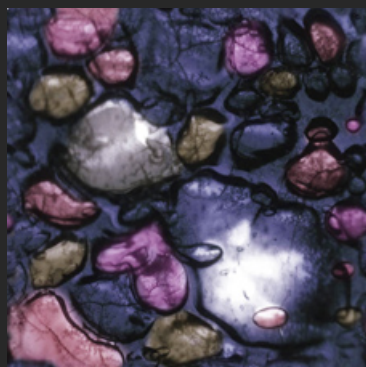


Inigo Quilez :: articles :: texture repetition - 2015

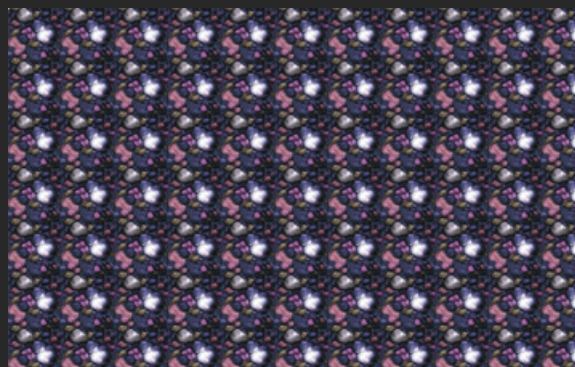


Problem

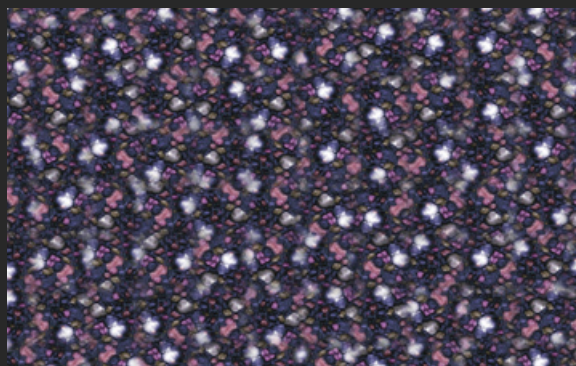
As of 2015, one of the most typical issues of using textures to cover large surfaces is the visible repetition of the textures. Texture warping modes like `GL_ARB_texture_mirrored_repeat` can alleviate the problem a little bit by making the period of repetition twice longer, but it's often insufficient; the hardware cannot solve the problem on its own. However, if we are okay with paying the cost of more than a single texture fetch per sample, then one can invent a few decent ways to avoid texture repetition. I'm going to present 3 different techniques I've come up with, but you'll notice they can be easily extended or modified to suit your needs.



The texture to be tiled



Regular texture tiling with `GL_REPEAT`



The proposed solutions with "Technique 1"

Technique 1

One way to prevent the visual repetition of the texture is to assign a random offset and orientation to each tile of the repetition. We can do that by determining in which tile the current sample is, then creating a series of four pseudo-random values for the tile, and then using these to offset and re-orient the texture. Re-orientation can be something as simple as mirroring in x or y or both or rotating. Here I'll use simple mirroring. Regardless of the particular transformation used, this tile based variation produces a non repeating pattern over the whole surface. You can see the result in the screenshots at the top of this article.

The technique just described comes with some caveats that need to be solved: First, the pattern will show seams across the tile boundaries, since the differently offsetted texture tiles won't match at the tile borders. Secondly, because of the discontinuity introduced on the final texture fetch coordinates themselves, the derivatives will have huge jumps at the tile borders and mipmapping will break apart, creating line artifacts.

One solution to solve both problems is to sample the texture with the offset and orientation mentioned above at four texture tiles, and blend between them when sufficiently close to the border of the current tile (in the positive U and V directions for example). While this will introduced some amount of blurring in the certain areas of the tile, it is acceptable in most cases, as shown in the image at the beginning of the article.

Of course, for this to work we must use custom texture gradients of course, which must come from the original repeating UV mapping.

The code is pretty simple, and you can find it live in Shadertoy: <https://www.shadertoy.com/view/lt2GDd>

```
vec4 textureNoTile( sampler2D samp, in vec2 uv )
{
    ivec2 iuv = ivec2( floor( uv ) );
    vec2 fuv = fract( uv );

    // generate per-tile transform
    vec4 ofa = hash4( iuv + ivec2(0,0) );
    vec4 ofb = hash4( iuv + ivec2(1,0) );
    vec4 ofc = hash4( iuv + ivec2(0,1) );
    vec4 ofd = hash4( iuv + ivec2(1,1) );

    vec2 ddx = dFdx( uv );
    vec2 ddy = dFdy( uv );
```

```

// transform per-tile uvs
ofa.zw = sign( ofa.zw-0.5 );
ofb.zw = sign( ofb.zw-0.5 );
ofc.zw = sign( ofc.zw-0.5 );
ofd.zw = sign( ofd.zw-0.5 );

// uv's, and derivatives (for correct mipmapping)
vec2 uva = uv*ofa.zw + ofa.xy, ddx_a = ddx*ofa.zw, ddy_a = ddy*ofa.zw;
vec2 uvb = uv*ofb.zw + ofb.xy, ddx_b = ddx*ofb.zw, ddy_b = ddy*ofb.zw;
vec2 uvc = uv*ofc.zw + ofc.xy, ddx_c = ddx*ofc.zw, ddy_c = ddy*ofc.zw;
vec2 uvd = uv*ofd.zw + ofd.xy, ddx_d = ddx*ofd.zw, ddy_d = ddy*ofd.zw;

// fetch and blend
vec2 b = smoothstep( 0.25,0.75, fuv );

return mix( mix( textureGrad( samp, uva, ddx_a, ddy_a ),
                  textureGrad( samp, uvb, ddx_b, ddy_b ), b.x ),
            mix( textureGrad( samp, uvc, ddx_c, ddy_c ),
                  textureGrad( samp, uvd, ddx_d, ddy_d ), b.x ), b.y );
}

```

Note that the code propagates the orientation mirror transformation to the derivatives. Since the underlying hardware is probably taking the absolute value of these, you can pretty safely optimize those away and simply pass **ddx** and **ddy** to the **textureGrad()** function.

The only remaining caveat with this technique is that the per-tile hash function might alias at high minification factors. For example, if this technique is used to texture a huge terrain, depending on the way this texturing method is used, aliasing might occur in the horizon or distant parts of the terrain.

Needles to say this technique can be used not only with squares, but also with any pattern that tiles space, such as triangles or hexagons.

Technique 2

Another way to get even more organic looking texture un-tile-fication (just invented a word there) is to bomb the whole surface with randomly scaled, offseted and rotated copies of the original texture which get blended together, with the blending weight factor dependent on the distance to the center of each of these copies. This can be accomplished with a **smooth voronoi** pattern for example. Blending weights proportional to a gaussian fall off for each feature point in the voronoi pattern works fine. Just remember to renormalize the final color to

the total contribution of each feature point, otherwise texture brightness range will be lost.

Live code in Shadertoy can be reached here: <https://www.shadertoy.com/view/4tsGzf>

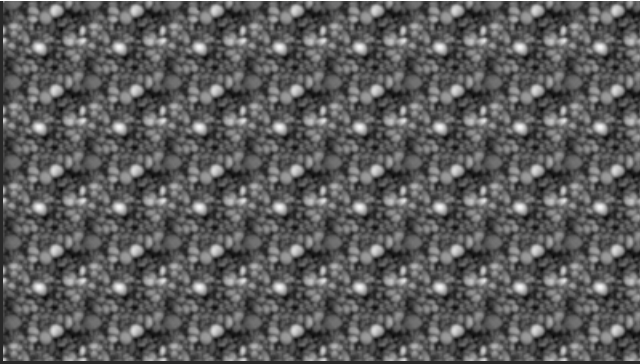
```
vec4 textureNoTile( sampler2D samp, in vec2 uv )
{
    vec2 p = floor( uv );
    vec2 f = fract( uv );

    // derivatives (for correct mipmapping)
    vec2 ddx = dFdx( uv );
    vec2 ddy = dFdy( uv );

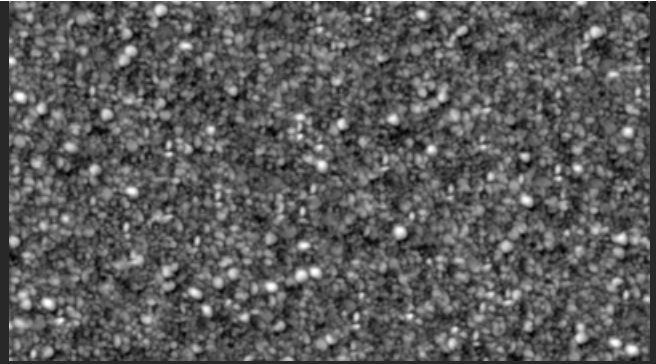
    // voronoi contribution
    vec4 va = vec4( 0.0 );
    float wt = 0.0;
    for( int j=-1; j<=1; j++ )
        for( int i=-1; i<=1; i++ )
        {
            vec2 g = vec2( float(i), float(j) );
            vec4 o = hash4( p + g );
            vec2 r = g - f + o.xy;
            float d = dot(r,r);
            float w = exp(-5.0*d );
            vec4 c = textureGrad( samp, uv + o.zw, ddx, ddy );
            va += w*c;
            wt += w;
        }

    // normalization
    return va/wt;
}
```

Of course, the drawback is the algorithm samples the texture 9 times, which might stress the memory bus too much. But in the other hand, it really help with high quality imagery or situation where just simply can afford it.



Regular texture tiling with GL_REPEAT



Smooth Voronoi based tiling

Technique 3

There's a really cheap way to achieve this as well with a different concept. The idea is to not have tiles like in Technique 1 but regions similar to Technique 2, but defined differently. First, let's start by letting the texture repeat over the plane as usual. Then, imagine we have several virtual versions of this tiling pattern, say 8, by simply applying constant offsets to the texture lookup. One can make the technique more powerful by allowing rotations, symmetries and scales for these 8 virtual versions, but for our purposes offsets suffice most of the time. Now, the final un-repeating pattern is evaluated at each point of the texturing domain by first picking one number between 0 and 7, which we can call *index*, and then picking one of these versions for sampling the texels based on it. By choosing the same values for the index within regions, we can create patches of the plane that use the same virtual pattern. Of course seams would be visible, so in order to improve this we actually make the index a floating point value instead of an integer. That way the index can change slowly and smoothly over the plane. We can then use it to interpolate between the two closest virtual patterns rather than just picking one. This low frequency index variation pattern can be procedural noise, or random values coming from a LookUp Table or a texture, which makes it easier to filter and therefore get a completely filterable resulting pattern (unlike index values based on a procedural function which is usually more difficult to filter).

The code for the technique is below, and some live code in Shadertoy can be found here:

<https://www.shadertoy.com/view/Xtl3zf>

```
vec4 textureNoTile( sampler2D samp, in vec2 uv )
{
    // sample variation pattern
    float k = texture( iChannel1, 0.005*x ).x; // cheap (cache friendly) lookup

    // compute index
```

```

float index = k*8.0;
float i = floor( index );
float f = fract( index );

// offsets for the different virtual patterns
vec2 offa = sin(vec2(3.0,7.0)*(i+0.0)); // can replace with any other hash
vec2 offb = sin(vec2(3.0,7.0)*(i+1.0)); // can replace with any other hash

// compute derivatives for mip-mapping
vec2 dx = dFdx(x), dy = dFdy(x);

// sample the two closest virtual patterns
vec3 cola = textureGrad( iChannel0, x + offa, dx, dy ).xxx;
vec3 colb = textureGrad( iChannel0, x + offb, dx, dy ).xxx;

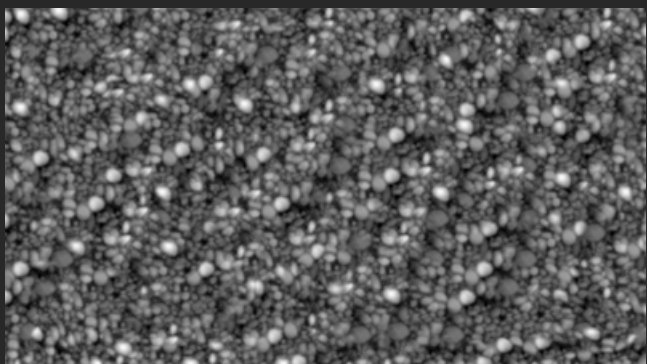
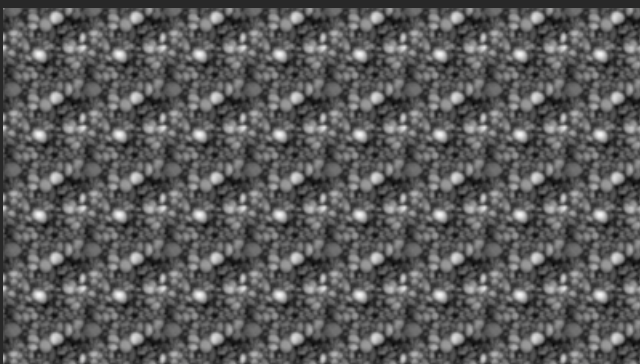
float sum( vec3 v ) { return v.x+v.y+v.z; }

// interpolate between the two virtual patterns
return mix( cola, colb, smoothstep(0.2,0.8,f-0.1*sum(cola-colb)) );
}

```

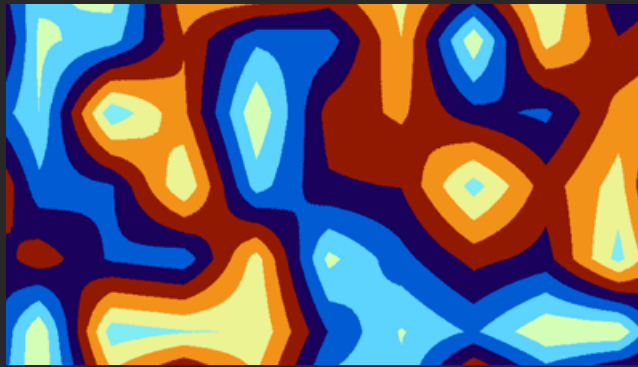
Several notes here: First, provided the variation pattern is low frequency, sampling it is really cheap since almost all the times the data will be in the texture cache. Second, the hashes for creating the offsets for the virtual patterns can be as sophisticated as needed. Third, texture coordinate derivatives need to be computed for proper filtering since we are introducing discontinuities. Forth, the interpolation function can be a cubic or linear like in this example above, and can be enriched with any other fancy technique that makes the images look good - in this example case I'm using the difference between the two virtual patterns to boost the contrast of the blended areas.

Bellow is a comparison of the regular tiling pattern and the Technique 3, which is only two texture fetches, together with a break down of the regions in which the different virtual textures are sampled:



Regular texture tiling with GL_REPEAT

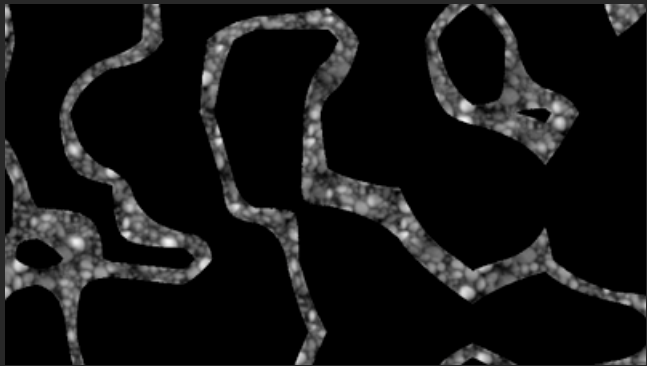
Technique 3 in action



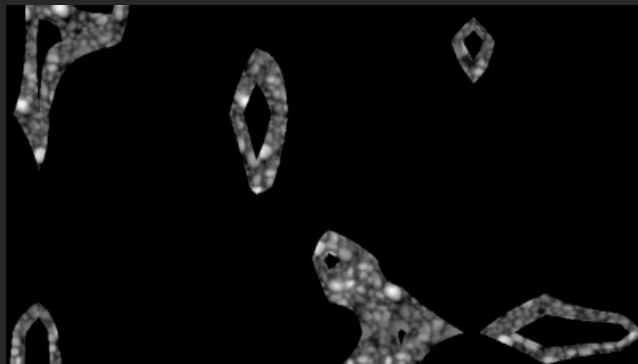
The index used for variation, encoded as color



Variation 2, masked by index



Variation 4, masked by index



Variation 6, masked by index

inigo quilez - learning computer graphics since 1994